

Лекция 6. Работа с базами данных

- Базы данных SQLite.
- Язык запросов.
- Намерения (явные, неявные).
- Библиотеки Retrofit, Room, Moshi.

Базы данных SQLite

SQLite — это встроенная в Android система управления базами данных, основанная на SQL. Она позволяет хранить структурированные данные локально, что особенно важно для приложений, требующих работы без интернета или с большими объёмами данных. В Android SQLite реализуется через Room — абстрактный слой, упрощающий работу с базой данных и добавляющий безопасность типов. Ключевые компоненты Room следующие:

- Entity — класс, представляющий таблицу базы данных.
- DAO (Data Access Object) — интерфейс, описывающий методы для операций вроде вставки, чтения, обновления.
- Database — абстрактный класс, объединяющий Entity и DAO.

Пример: Сохранение данных о пользователях через Room.

```
// 1. Entity: сущность User
@Entity(tableName = "user_table")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int,
    @ColumnInfo(name = "name") val name: String,
    @ColumnInfo(name = "email") val email: String
)

// 2. DAO: методы для работы с данными
@Dao
interface UserDao {
    @Insert
    suspend fun insert(user: User)

    @Query("SELECT * FROM user_table")
    fun getAllUsers(): LiveData<List<User>>
```

```

}
// 3. Database: подключение к базе данных
@Database(entities = [User::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
// 4. Инициализация базы данных
val database = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "user-database"
).build()
// 5. Использование DAO для записи данных
val userDao = database.userDao()
userDao.insert(User(id = 0, name = "Alice", email = "alice@example.com"))

```

Здесь Room автоматически генерирует SQL-запросы на основе аннотаций (@Insert, @Query и т.д.). LiveData обеспечивает реактивное обновление UI при изменении данных., suspend указывает, что метод должен выполняться в корутине, чтобы не блокировать основной поток.

Язык запросов

Работа с базами данных в Android строится на SQL-подобном языке запросов. Room предоставляет два способа работы с данными:

- Аннотации (@Insert, @Update, @Delete) — для стандартных операций.
- Пользовательские SQL-запросы через @Query — для сложных сценариев.

SQL-запросы в Room должны быть статичными и безопасными, так как они проверяются на этапе компиляции. Это исключает ошибки при запуске.

Пример: Получение пользователей с определённым именем.

```

@Dao
interface UserDao {
    // Использование SQL-запроса для фильтрации
    @Query("SELECT * FROM user_table WHERE name = :name")
    fun getUsersByName(name: String): LiveData<List<User>>
}

```

// Использование:

```
val users = userDao getUsersByName("Alice")
```

Здесь `@Query` позволяет использовать сложные SQL-выражения, такие как `JOIN`, `GROUP BY`, `LIMIT`, `LiveData` автоматически обновляет UI при изменении данных, `Room` поддерживает асинхронные запросы через `suspend` и корутины, что предотвращает блокировку UI-потока.

Намерения (Intent)

Намерения (Intent) — это механизм передачи данных между компонентами Android (например, между активностями или фрагментами). Они бывают двух типов:

- Явные намерения: Указывают конкретный класс для запуска.
- Неявные намерения: Описывают действие, которое должно быть выполнено, без указания конкретного получателя.

Intent часто используются для передачи данных, например, из базы данных в другую активность.

Пример: Передача данных пользователя через явное намерение.

```
// Получение данных из базы
val user = userDao.getUserById(1)

// Создание намерения для перехода к активности профиля
val intent = Intent(this, ProfileActivity::class.java).apply {
    putExtra("USER_ID", user.id)
    putExtra("USER_NAME", user.name)
}
startActivity(intent)
```

```
// В ProfileActivity:
val intent = intent
val userId = intent.getIntExtra("USER_ID", -1)
val userName = intent.getStringExtra("USER_NAME")
Здесь putExtra() — передаёт данные в намерении.
getStringExtra() — извлекает данные в другой активности.
```

Для сложных объектов можно использовать `Parcelable` или `Serializable`.

Библиотеки Retrofit, Room, Moshi

Для работы с данными в Android используется экосистема библиотек, каждая из которых решает свою задачу:

- Retrofit — для работы с сетью и API.
- Room — для локального хранения данных.
- Moshi — для парсинга JSON в Kotlin-объекты и обратно.

Эти библиотеки часто работают вместе: данные загружаются через Retrofit, парсятся с помощью Moshi и сохраняются в Room для кэширования.

Пример: Загрузка данных с сервера и сохранение в Room.

```
// 1. Retrofit: загрузка данных
interface ApiService {
    @GET("users")
    suspend fun fetchUsers(): Response<List<User>>
}

// 2. Moshi: автоматический парсинг JSON
val moshi = Moshi.Builder().build()
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com ")
    .addConverterFactory(MoshiConverterFactory.create(moshi))
    .build()

// 3. Room: сохранение в локальную базу
val apiService = retrofit.create(ApiService::class.java)
val userDao = database.userDao()

// 4. Загрузка и сохранение
val response = apiService.fetchUsers()
if (response.isSuccessful) {
    response.body()?.forEach { user ->
        userDao.insert(user)
    }
}
```

Здесь Retrofit использует suspend-функции для работы в корутинах, обеспечивая асинхронность. Moshi автоматически конвертирует JSON в Kotlin-объекты, упрощая работу с API. Room хранит данные локально, снижая частоту сетевых запросов и повышая отзывчивость приложения.

Интеграция всех элементов в приложение

Работа с данными в Android-приложениях требует тесного взаимодействия между локальным хранением, сетевыми запросами и передачей информации внутри приложения. Эта интеграция строится на комбинации баз данных, сетевых библиотек и механизмов межкомпонентного взаимодействия, которые вместе обеспечивают стабильность, отзывчивость и адаптивность приложения к различным условиям использования.

SQLite — это встроенная в Android система управления базами данных, которая позволяет хранить информацию на устройстве без зависимости от интернета. Однако прямая работа с SQLite требует ручного написания SQL-запросов, что повышает вероятность ошибок. Для упрощения и обеспечения безопасности типов используется Room, который добавляет уровень абстракции над SQLite. Room автоматически генерирует SQL-запросы на основе аннотаций, таких как `@Insert`, `@Update`, `@Query`, и обеспечивает реактивность через LiveData, что позволяет интерфейсу обновляться при изменениях в базе данных. Например, если приложение отображает список пользователей, то при добавлении нового пользователя в базу данных, UI автоматически обновится, если используется LiveData.

Intent — это механизм, который связывает данные из базы с интерфейсом. Он позволяет передавать информацию между активностями или фрагментами, например, при переходе к экрану деталей записи. Intent может передавать примитивные типы (строки, числа), а также сложные объекты, реализующие интерфейс `Parcelable` или `Serializable`. Это особенно полезно, когда данные уже загружены из Room и их нужно отобразить в другой активности без повторного запроса к базе.

Интеграция Room, SQL-запросов, Intent и сетевых библиотек (Retrofit, Moshi) формирует основу для эффективного управления данными в Android-приложении. Room обеспечивает локальное хранение, SQL-запросы через `@Query` делают обработку данных безопасной и читаемой, Intent связывает информацию с интерфейсом, а Retrofit с Moshi — загружает данные из сети и преобразует их в структурированные объекты. Эти инструменты работают

вместе, создавая надёжную и производительную систему хранения и обработки данных.